

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Gabriela Pricop, FAF-241

Report

Laboratory Work No. 5

Operating Systems

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

1 Part 1: The tree and the file types **3**

2 Part 2: Inodes and links **5**

3 Part 3: Permissions **9**

4 Part 4: Disks and the kernel as files **12**

Part 1: The tree and the file types

Questions:

- For each path of the command `ls -ld /etc /etc/passwd /bin /dev/null /tmp`, write its type from the first letter of `ls -l`.
- What is `/bin` on this system: a directory or a link, and to where?
- Find one character device and one block device in `/dev`.
- Which is bigger, `/var/log` or `/usr`, and by roughly how much?

Answers:

- The first letters in Figure 1.1 are: `/etc` is `d`, a directory; `/etc/passwd` is `-`, a regular file; `/bin` is `l`, a symbolic link; `/dev/null` is `c`, a character device; and `/tmp` is `d`, a directory. The `t` at the end of the mode of `/tmp` is the sticky bit, which does not change the type. The result is summarized in Table 1.
- `/bin` is a symbolic link and not a directory: its line begins with `l` and ends with `/bin -> usr/bin`. The size of the link, 7 bytes, is the length of the text `usr/bin` stored in it, and `file` confirms the result with `symbolic link to usr/bin`.
- `/dev/null` is a character device (`c`, with the major number 1 and the minor number 3 written in the place of the size), and so are `autofs`, `btrfs-control` and `console` in Figure 1.2. None of the first ten lines is a block device (`block` is only a directory), but `lsblk` in Part 4 lists the block devices of this system, for example `/dev/loop0` (major 7, minor 0) and `/dev/sr0`, the device of the installation disc, which `df` shows mounted on `/cdrom`. In `ls -l /dev` such entries begin with `b`.
- `/usr` is much bigger: 5.3G against 21M for `/var/log`, a difference of about 5.3 GB, so `/usr` is roughly 250 times bigger.

```
ubuntu@ubuntu:~/Desktop$ whoami
ubuntu
ubuntu@ubuntu:~/Desktop$ mkdir -p ~/os-lab5 && cd ~/os-lab5

ubuntu@ubuntu:~/os-lab5$ ls /
bin   cdrom  etc   lib    media  opt   rofs  run   snap  sys  usr
boot  dev    home  lib64  mnt    proc  root  sbin  srv   tmp  var
ubuntu@ubuntu:~/os-lab5$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx  1 root root    7 Apr 20 08:46 /bin -> usr/bin
crw-rw-rw-  1 root root 1, 3 Oct 12 14:35 /dev/null
drwxr-xr-x  1 root root  740 Oct 12 14:35 /etc
-rw-r--r--  1 root root 2856 Oct 12 14:35 /etc/passwd
drwxrwxrwt 17 root root  400 Oct 12 16:02 /tmp
ubuntu@ubuntu:~/os-lab5$ S
```

Figure 1.1: The working directory, the top of the tree and the types of five paths

whoami – prints the name of the current user. The result `ubuntu` (Figure 1.1) proves

that the work is done as a normal user and not as root, so the permission checks of the next parts are applied.

mkdir -p ~/os-lab5 && cd ~/os-lab5 – creates the working directory (the option `-p` does not fail if the directory already exists) and enters it only if the creation succeeded (`&&`). The prompt in Figure 1.1 changes from `~/Desktop` to `~/os-lab5`.

ls / – lists the top level of the tree. Besides the standard directories (`bin`, `boot`, `dev`, `etc`, `home`, `lib`, `proc`, `root`, `run`, `sys`, `tmp`, `usr`, `var`) it shows `cdrom`, `rofs` and `snap`, which are present because the system runs from an installation image. The name `tmp` is highlighted, because `ls` marks in this way a directory that everybody can write and that has the sticky bit.

ls -ld /etc /etc/passwd /bin /dev/null /tmp – the option `-l` gives the long format and `-d` shows the entries themselves and not the contents of the directories. The first character of every line is the file type: `d` for `/etc` and `/tmp`, `-` for `/etc/passwd`, `l` for `/bin` and `c` for `/dev/null`. For `/dev/null` the size is replaced by the pair `1, 3`, the major and the minor number of the device.

```
ubuntu@ubuntu:~/os-lab5$ ls -l /dev | head
total 0
crw-r--r--  1 root   root     10, 235 Oct 12 14:35 autofs
drwxr-xr-x  2 root   root         500 Oct 12 15:00 block
drwxr-xr-x  2 root   root          80 Oct 12 14:34 bsg
crw-rw----  1 root   disk     10, 234 Oct 12 14:33 btrfs-control
drwxr-xr-x  3 root   root          60 Oct 12 14:35 bus
lrwxrwxrwx+ 1 root   root           3 Oct 12 14:34 cdrom -> sr0
drwxr-xr-x  2 root   root        3820 Oct 12 15:42 char
crw-------  1 root   root      5,   1 Oct 12 14:35 console
lrwxrwxrwx  1 root   root          11 Oct 12 14:35 core -> /proc/kcore
ubuntu@ubuntu:~/os-lab5$ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: symbolic link to ../lib/cargo/bin/coreutils/ls
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin
ubuntu@ubuntu:~/os-lab5$
```

Figure 1.2: The first lines of `ls -l /dev` and the result of `file`

ls -l /dev | head – shows the first ten lines of the long listing of `/dev`. The entries `autofs`, `btrfs-control` and `console` begin with `c` (character devices), `block`, `bsg`, `bus` and `char` begin with `d` (directories), and `cdrom -> sr0` and `core -> /proc/kcore` begin with `l` (symbolic links). No `b` appears in these ten lines.

file /usr/bin/ls /etc/passwd /bin – determines the type from the content and not from the mode. In Figure 1.2, `/usr/bin/ls` and `/bin` are symbolic links, the first to `../lib/cargo/bin/coreutils/ls` and the second to `usr/bin`, and `/etc/passwd` is `ASCII text`, that is, a plain text file.

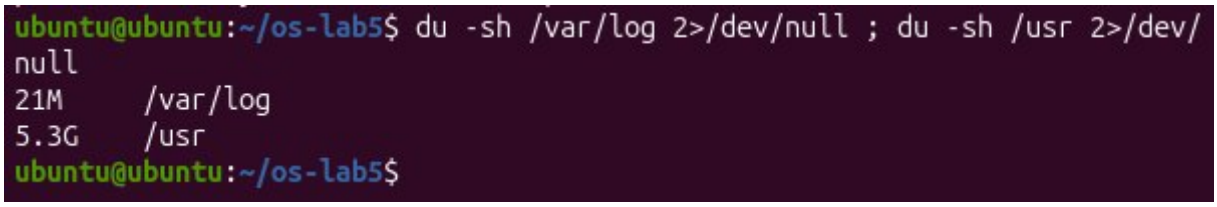


Figure 1.3: The sizes of `/var/log` and `/usr`

du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null – `du -s` prints only the total of each directory and `-h` writes it in readable units. The redirection `2>/dev/null` throws away the error messages (descriptor 2) of the directories that a normal user cannot read, so the totals can be slightly smaller than the real ones. Figure 1.3 shows 21M for `/var/log` and 5.3G for `/usr`.

Path	**First letter**	**Type**	**Shown in**
`/etc`	`d`	directory	Figure 1.1
`/etc/passwd`	`-`	regular file	Figure 1.1
`/bin`	`l`	symbolic link to `usr/bin`	Figure 1.1
`/dev/null`	`c`	character device (1, 3)	Figure 1.1
`/tmp`	`d`	directory with the sticky bit	Figure 1.1

Table 1: The types of the paths of `ls -ld`

Table 1 shows that the type of a file is given by the first letter of the mode and not by its name or its place in the tree. Even `/bin`, the directory in which the programs were traditionally kept, is only a link to `/usr/bin` on this system.

Part 2: Inodes and links

Questions:

- Which number in `stat` is the inode, and which tells how many names the file has?
- After `rm notes.txt`, why could you still read `hard.txt`? What does `rm` really remove?
- Why did `cat soft.txt` fail at the end, although `soft.txt` still exists? How big is `soft.txt` and why exactly that size?
- A new directory has 2 links, and 4 after you create two subdirectories. Explain where each link comes from.
- Why does a 2-byte file use 4.0K, and a 1 GB file use 0?

Answers:

- The inode number is the field `Inode: 19896` of `stat`, and it is also the first column of `ls -li`. The number of names of the file is the field `Links: 1`, which is the number after the mode in `ls -l`. The inode stores the owner, the mode, the size,

the times and the place of the data, but not the name; the names are entries of directories that connect a name to an inode number.

- After `ln notes.txt hard.txt` the inode 19896 had two names and the link count was 2. The command `rm notes.txt` removed only the name `notes.txt` and decreased the count to 1, so `hard.txt` still pointed to the same inode and `cat` printed `hello`. The command `rm` removes a name (it calls `unlink`); the inode and the data are released only when the last name is gone and no process keeps the file open.
- The file `soft.txt` is a symbolic link, a separate file with its own inode (20045) whose content is only the path of the target. After `rm hard.txt` the name `hard.txt` did not exist any more, so `cat` followed the path and failed with `No such file or directory`, while the link itself still existed (`ls -l` shows it in red). Its size is 8 bytes, because the stored text `hard.txt` has exactly 8 characters.
- The link count of `d` was 2 for two names: the entry `d` in the parent directory and the entry `.` inside `d`. Every subdirectory contains the entry `..`, which points back to `d`, so `d/x` and `d/y` added two more links and the count became 4. Ordinary files do not change the count of a directory, therefore the count is 2 plus the number of subdirectories.
- The file system allocates space in blocks (`IO Block: 4096` in `stat`), so the 2 bytes of `one.txt` (`x` and the newline) occupy one whole block of 4.0K, while `ls` shows the logical size of 2 bytes. The command `truncate -s 1G` only sets the size of `big.img` in the inode and creates a sparse file: no data block is allocated, the missing part is read as zeros, and `du`, which counts the allocated blocks, shows 0 although `ls -lh` shows 1.0G.

```
ubuntu@ubuntu:~/os-lab5$ echo hello > notes.txt
ubuntu@ubuntu:~/os-lab5$ stat notes.txt
  File: notes.txt
  size: 6               Blocks: 8          IO Block: 4096   regular file
Device: 0,29    Inode: 19896       Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/  ubuntu)   Gid: ( 1000/  ubuntu)
Access: 2026-10-12 16:16:23.569541912 +0000
Modify: 2026-10-12 16:16:23.488292500 +0000
Change: 2026-10-12 16:16:23.488292500 +0000
 Birth: 2026-10-12 16:16:23.488292500 +0000
ubuntu@ubuntu:~/os-lab5$ ls -li notes.txt
19896 -rw-rw-r-- 1 ubuntu ubuntu 6 Oct 12 16:16 notes.txt
ubuntu@ubuntu:~/os-lab5$
```

Figure 2.1: The inode of `notes.txt` in `stat` and `ls -li`

echo hello > notes.txt – creates the file `notes.txt` with the line `hello`. The size of the file is 6 bytes: five letters and the newline written by `echo`.

stat notes.txt – prints the information stored in the inode. Figure 2.1 shows `Size: 6`, `Blocks: 8` (units of 512 bytes, i.e. one block of 4096 bytes), `IO Block: 4096`, the type `regular file`, `Inode: 19896`, `Links: 1`, the mode `0664/-rw-rw-r--`, the owner `ubuntu` (uid and gid 1000) and the four times, which are all equal because the file was

just created.

ls -li notes.txt – the option `-i` adds the inode number as the first column. The number 19896 is the same as in `stat`, and the count 1 follows the mode.

```
ubuntu@ubuntu:~/os-lab5$ ls -li notes.txt
19896 -rw-rw-r-- 1 ubuntu ubuntu 6 Oct 12 16:16 notes.txt
ubuntu@ubuntu:~/os-lab5$ ln notes.txt hard.txt
ubuntu@ubuntu:~/os-lab5$ ls -li notes.txt hard.txt
19896 -rw-rw-r-- 2 ubuntu ubuntu 6 Oct 12 16:16 hard.txt
19896 -rw-rw-r-- 2 ubuntu ubuntu 6 Oct 12 16:16 notes.txt
ubuntu@ubuntu:~/os-lab5$
```

Figure 2.2: A hard link: two names with the same inode

ln notes.txt hard.txt ; ls -li notes.txt hard.txt – creates a second name for the same file. In Figure 2.2 both lines have the inode 19896, the same size, the same time and the link count 2, which proves that `hard.txt` is not a copy but another name of the same file.

```
ubuntu@ubuntu:~/os-lab5$ rm notes.txt
ubuntu@ubuntu:~/os-lab5$ cat hard.txt ; ls -li hard.txt
hello
19896 -rw-rw-r-- 1 ubuntu ubuntu 6 Oct 12 16:16 hard.txt
ubuntu@ubuntu:~/os-lab5$
```

Figure 2.3: The file is still readable after removing the original name

rm notes.txt ; cat hard.txt ; ls -li hard.txt – the first name is removed, but `cat` still prints `hello`, and the count of the inode 19896 is again 1 (Figure 2.3). The data was not deleted, because one name remained.

```
ubuntu@ubuntu:~/os-lab5$ ln -s hard.txt soft.txt
ubuntu@ubuntu:~/os-lab5$ ls -li hard.txt soft.txt
19896 -rw-rw-r-- 1 ubuntu ubuntu 6 Oct 12 16:16 hard.txt
20045 lrwxrwxrwx 1 ubuntu ubuntu 8 Oct 12 16:22 soft.txt -> hard.txt
ubuntu@ubuntu:~/os-lab5$
```

Figure 2.4: A symbolic link has its own inode

ln -s hard.txt soft.txt ; ls -li hard.txt soft.txt – creates a symbolic link. In Figure 2.4 the link `soft.txt` has a different inode (20045), the type `l`, the mode `lrwxrwxrwx`, the size 8 and the text `-> hard.txt` showing its target, while `hard.txt` keeps the inode 19896 and the size 6.

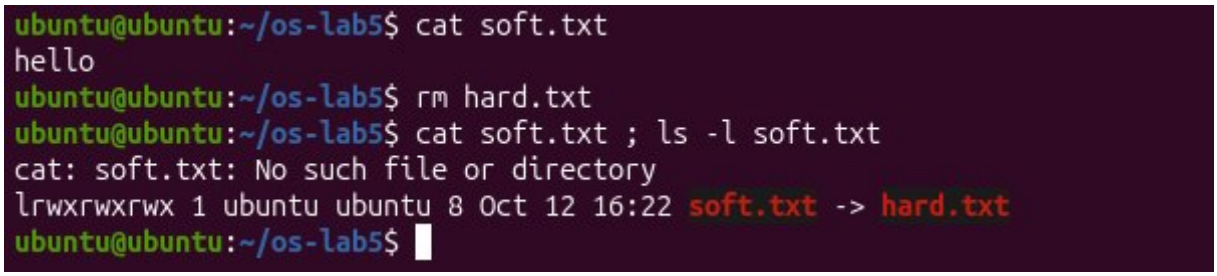


Figure 2.5: A symbolic link after the removal of its target

cat soft.txt ; rm hard.txt ; cat soft.txt ; ls -l soft.txt – while `hard.txt` exists, `cat soft.txt` follows the link and prints `hello`. After `rm hard.txt` the target is gone and `cat` prints `No such file or directory` (Figure 2.5), but `ls -l` still lists `soft.txt -> hard.txt`, now highlighted in red as a broken link.

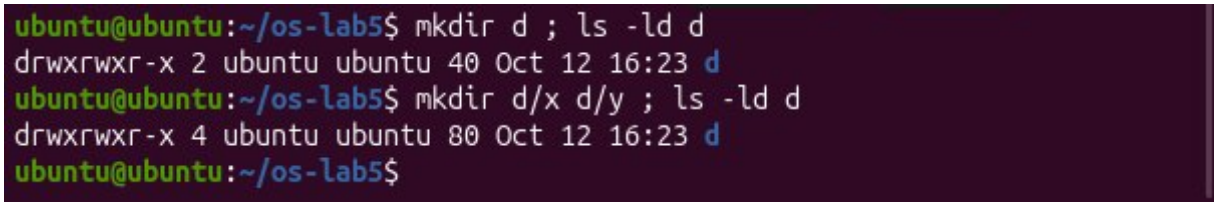


Figure 2.6: The link count of a directory before and after two subdirectories

mkdir d ; ls -ld d and **mkdir d/x d/y ; ls -ld d** – a new directory has the count 2, and after the creation of `d/x` and `d/y` the count is 4 (Figure 2.6), as explained in the answers.

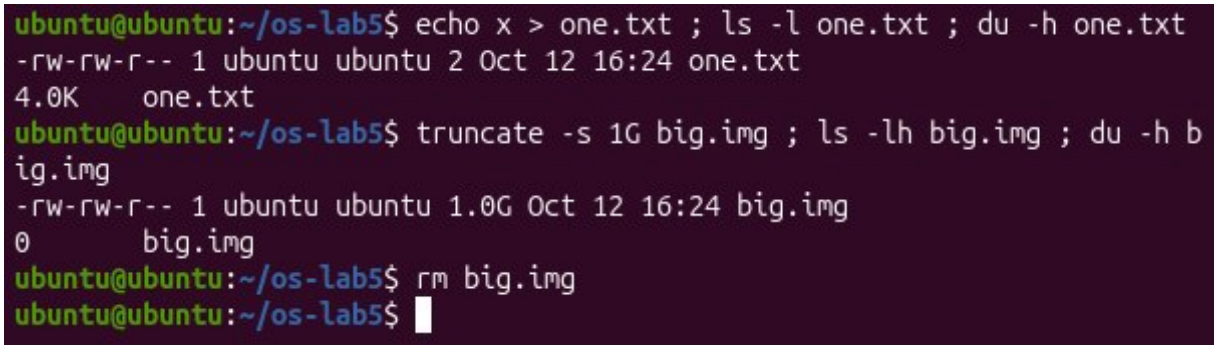


Figure 2.7: The size of a file and the space it uses

echo x > one.txt ; ls -l one.txt ; du -h one.txt – `ls` shows 2 bytes and `du` shows 4.0K, the whole block that was allocated (Figure 2.7).

truncate -s 1G big.img ; ls -lh big.img ; du -h big.img – `ls` shows the size 1.0G and `du` shows 0, because the file is sparse and has no allocated blocks. The command `rm big.img` finishes at once and returns no space, as none was used.

Property	**Hard link** (`hard.txt`)	**Symbolic link** (`soft.txt`)
Inode	19896, the same as the original name	20045, its own inode
Type in `ls -l`	`-`, regular file	`l`, symbolic link
Size	6 bytes, the data `hello`	8 bytes, the text `hard.txt`
Link count	2, then 1 after `rm notes.txt`	1
Content	the data of the file	the path of the target
After removing the original or the target	`cat` still prints `hello`	`cat: No such file or directory`

Table 2: Comparison of the hard link and the symbolic link

The results of this part are summarized in Table 2. A hard link is one more name of the same inode and keeps the data alive, while a symbolic link is a small file that stores a path and breaks when the path stops existing.

Part 3: Permissions

Questions:

- Write 600, 644, 755 and 700 as `rwx` strings, and give one real use for each.
- With `chmod 000` you own the file but cannot read it. Why does ownership not help?
- What is your `umask`, and how does it explain the permissions of `new.txt` and `newdir`?
- Explain the `box` experiment: what does `r` allow on a directory, and what does `x` allow?
- What do the `s` in `/usr/bin/passwd` and the `t` in `/tmp` mean, and why does `passwd` need the `s`?

Answers:

- The mode 600 is `rw-------`, used for private files, such as the file `s.txt` of this part or a private key in `~/.ssh`. The mode 644 is `rw-r--r--`, used for ordinary files that everybody may read, such as `/etc/passwd`. The mode 755 is `rwxr-xr-x`, used for programs and public directories, such as `/etc`. The mode 700 is `rwx------`, used for a private directory, such as `~/.ssh`. The values are collected in Table 3.
- The kernel checks the permissions at every opening and uses only one class: the owner class for the owner, then the group, then the others. For `s.txt` the owner class had no bits, so `cat` failed with `Permission denied`, even though the file belonged to the user. Ownership gives another right, the right to change the mode: `chmod u+r s.txt` worked and `cat` printed `secret` again (Figure 3.2).
- The `umask` is `0002`. A new file starts from 666 and a new directory from 777, and the bits of the mask are removed from them: $666 - 002 = 664$, which is `-rw-rw-r--`

for `new.txt`, and $777 - 002 = 775$, which is `drwxrwxr-x` for `newdir`. These are exactly the modes shown in Figure 3.2.

- The permission `r` on a directory allows reading the list of names, and `x` allows passing through the directory to reach an entry whose name is known (and entering it with `cd`). With `u=r` the command `ls box` printed `f`, but `cat box/f` failed, because reaching `f` needs `x` on `box`. With `u=x` the command `ls box` failed with `cannot open directory`, because listing needs `r`, but `cat box/f` printed `hi`, because `x` was present and the file `f` itself is readable (Figure 3.3).
- The `s` replaces the `x` of the owner in `-rwsr-xr-x` and is the setuid bit: the program runs with the identity of its owner, here root, and not with the identity of the user who starts it. The program `passwd` must change `/etc/shadow`, which is `-rw-r-----` and belongs to root and to the group `shadow`, so an ordinary user can neither read nor write it (Figure 3.4). Thanks to the `s`, `passwd` can update the password of the user although the user has no right to the file. The `t` replaces the `x` of the others in `drwxrwxrwt` and is the sticky bit: everybody may create files in `/tmp`, but only the owner of a file (or the owner of the directory, or root) may delete or rename it, so users cannot remove the files of each other.

```
ubuntu@ubuntu:~/os-lab5$ echo secret > s.txt
ubuntu@ubuntu:~/os-lab5$ chmod 600 s.txt ; ls -l s.txt
-rw------- 1 ubuntu ubuntu 7 Oct 12 16:25 s.txt
ubuntu@ubuntu:~/os-lab5$ chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied
ubuntu@ubuntu:~/os-lab5$
```

Figure 3.1: The modes 600 and 000 of the file `s.txt`

echo secret > s.txt – creates the file `s.txt` with 7 bytes, the word `secret` and the newline.

chmod 600 s.txt ; ls -l s.txt – the mode becomes `-rw-------`: only the owner can read and write (Figure 3.1).

chmod 000 s.txt ; cat s.txt – no class has any permission, and `cat` answers `Permission denied` although the file belongs to the user who runs it.

```
ubuntu@ubuntu:~/os-lab5$ chmod u+r s.txt ; cat s.txt
secret
ubuntu@ubuntu:~/os-lab5$ umask ; touch new.txt ; mkdir newdir ; ls -ld new
.txt newdir
0002
-rw-rw-r-- 1 ubuntu ubuntu  0 Oct 12 16:26 new.txt
drwxrwxr-x 2 ubuntu ubuntu 40 Oct 12 16:26 newdir
ubuntu@ubuntu:~/os-lab5$
```

Figure 3.2: Restoring the read permission and the effect of `umask`

chmod u+r s.txt ; cat s.txt – the owner adds the read permission to himself and `cat` prints `secret` (Figure 3.2).

umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir – the mask is `0002`. The new file has the mode `-rw-rw-r--` and the size 0, and the new directory has the mode `drwxrwxr-x`, as calculated in the answers.

```
ubuntu@ubuntu:~/os-lab5$ mkdir box ; echo hi > box/f
ubuntu@ubuntu:~/os-lab5$ chmod u=r,go= box  ; ls box ; cat box/f
f
cat: box/f: Permission denied
ubuntu@ubuntu:~/os-lab5$ chmod u=x,go= box  ; ls box ; cat box/f
ls: cannot open directory 'box': Permission denied
hi
ubuntu@ubuntu:~/os-lab5$ chmod 755 box
ubuntu@ubuntu:~/os-lab5$
```

Figure 3.3: The directory `box` with the permissions read only and execute only

mkdir box ; echo hi > box/f – creates a directory with the file `f` that contains `hi`.

chmod u=r,go= box ; ls box ; cat box/f – the owner keeps only `r`, and the group and the others lose all permissions. In Figure 3.3 `ls box` prints the name `f`, and `cat box/f` prints `Permission denied`.

chmod u=x,go= box ; ls box ; cat box/f – now the owner keeps only `x`. The command `ls box` prints `ls: cannot open directory 'box': Permission denied`, and `cat box/f` prints `hi`.

chmod 755 box – restores the normal mode of the directory, so that it can be used and removed later.

```
ubuntu@ubuntu:~/os-lab5$ ls -l /usr/bin/passwd ; ls -ld /tmp
-rwsr-xr-x 1 root root 93640 Feb  2  2026 /usr/bin/passwd
drwxrwxrwt 17 root root 400 Oct 12 16:27 /tmp
ubuntu@ubuntu:~/os-lab5$ cat /etc/shadow
cat: /etc/shadow: Permission denied
ubuntu@ubuntu:~/os-lab5$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1309 Oct 12 14:32 /etc/shadow
ubuntu@ubuntu:~/os-lab5$
```

Figure 3.4: The special bits of `/usr/bin/passwd` and `/tmp`, and the file `/etc/shadow`

ls -l /usr/bin/passwd ; ls -ld /tmp – the mode of `passwd` is `-rwsr-xr-x` with the owner root (setuid), and the mode of `/tmp` is `drwxrwxrwt` (sticky bit), as shown in Figure 3.4.

cat /etc/shadow ; ls -l /etc/shadow – the first command is refused with `Permission`

`denied`. The mode of the file is `-rw-r-----` with the owner root and the group `shadow`, and the user `ubuntu` is neither of them. The file with the password hashes is therefore protected from ordinary users and can be changed only through programs like `passwd`.

Mode	String	Real use
600	`rw-------`	private file: `s.txt`, a private key in `~/.ssh`
644	`rw-r--r--`	ordinary file that everybody may read: `/etc/passwd`
755	`rwxr-xr-x`	program or public directory: `/etc`, the directory `box` after `chmod 755`
700	`rwx------`	private directory: `~/.ssh`

Table 3: The four modes as `rwx` strings

Table 3 shows that every mode is three groups of three bits, for the owner, the group and the others, and that the same bits mean different things on files and on directories.

Part 4: Disks and the kernel as files

Questions:

- Draw your disk: disk name, partitions, which one is mounted at `/` and with what file system type.
- How full is your root file system?
- How many CPU cores and how much RAM does your laptop have, according to `/proc`?
- `/proc/self/status` showed the name of a program. Which one, and why that one?

Answers:

- The system runs as a live session from an installation image, and this is visible in the devices (Table 4). The command `lsblk` lists the devices `loop0` to `loop5` of the type `loop`, all read-only (`RO` is 1): `loop0` (3.2G) is mounted on `/rofs`, and `loop3`, `loop4` and `loop5` are the snap packages `core24`, `desktop-security-center` and `bare`. The device `/dev/sr0` (6.1G) holds the installation image and is mounted on `/cdrom`. The root directory is not mounted from a partition: `mount` shows `/cow on / type overlay`, so the file system type of `/` is `overlay`, built from read-only squashfs layers (`lowerdir`) and a writable layer `/cow/upper` (`upperdir`) in which the changes of the session are kept.
- The root file system is 13% full: `df -h` shows 3.9G of size, 478M used and 3.5G available for `/cow`, which is mounted on `/`. The size of 3.9G is about half of the RAM, which indicates that the writable layer is kept in memory.
- The command `grep -c processor /proc/cpuinfo` prints 4, because the file has one block for every logical processor, so the system has 4 CPUs. The first line of `/proc/meminfo` shows `MemTotal:  8126576 kB`, about 7.75 GiB, that is, 8 GB of

RAM. Only 233148 kB are `MemFree`, because the kernel uses free memory for caches, but `MemAvailable` is 4466992 kB, about 4.3 GiB, which can be given to programs.

- The name is `head` (`Name: head`). The path `/proc/self` always means the process that opens it, and the command that opened `/proc/self/status` was `head -5 /proc/self/status`, so the kernel generated the file for the process `head`. With `cat` the name would be `cat`. The other lines describe the same process: `State: R (running)`, because it was running while it read the file, `Umask: 0002`, the same value as in Part 3, and `Tgid: 18580`, its process number.

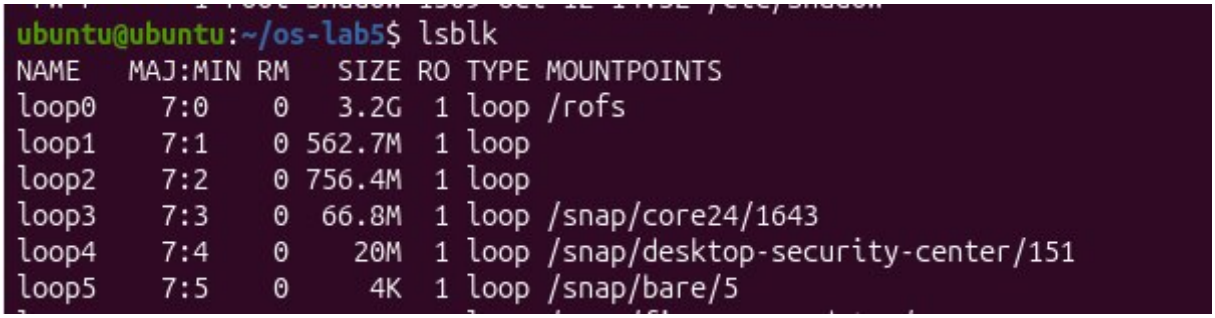


Figure 4.1: The block devices in `lsblk`

lsblk – lists the block devices with the columns `NAME`, `MAJ:MIN`, `RM`, `SIZE`, `RO`, `TYPE` and `MOUNTPOINTS`. Figure 4.1 shows the loop devices: `loop0` of 3.2G is mounted on `/rofs`, `loop1` (562.7M) and `loop2` (756.4M) have no mount point, and `loop3` (66.8M), `loop4` (20M) and `loop5` (4K) are mounted in `/snap`. The major number 7 is the number of the loop driver.

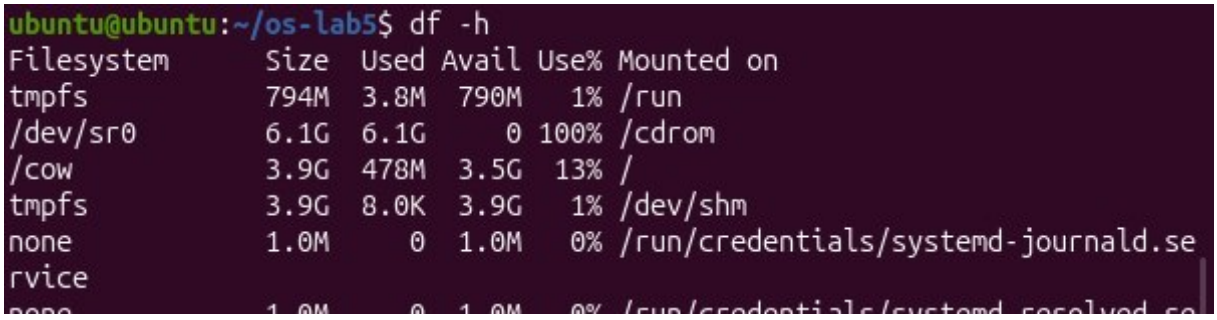


Figure 4.2: The mounted file systems and their use in `df -h`

df -h – shows the size, the used and the available space of the mounted file systems in readable units. In Figure 4.2 `/dev/sr0` is 100% used and mounted on `/cdrom`, `/cow` (3.9G, 478M used, 13%) is mounted on `/`, and the rest are `tmpfs` file systems in memory, such as `/run` and `/dev/shm`.

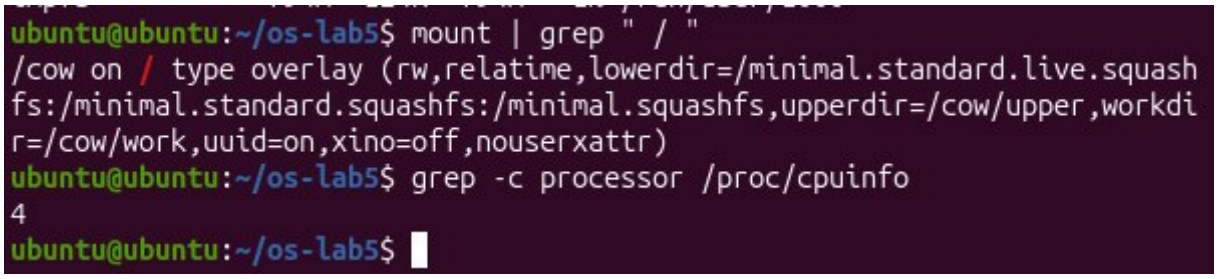


Figure 4.3: The root file system in `mount` and the number of processors

mount | grep " / " – selects the line of the root file system. Figure 4.3 shows `/cow on / type overlay` with the options `rw`, `relatime`, the read-only layers `lowerdir` (squashfs images), the writable layer `upperdir=/cow/upper` and the work directory `workdir=/cow/work`.

grep -c processor /proc/cpuinfo – counts the lines that contain the word `processor`, one for each logical processor. The result is 4.

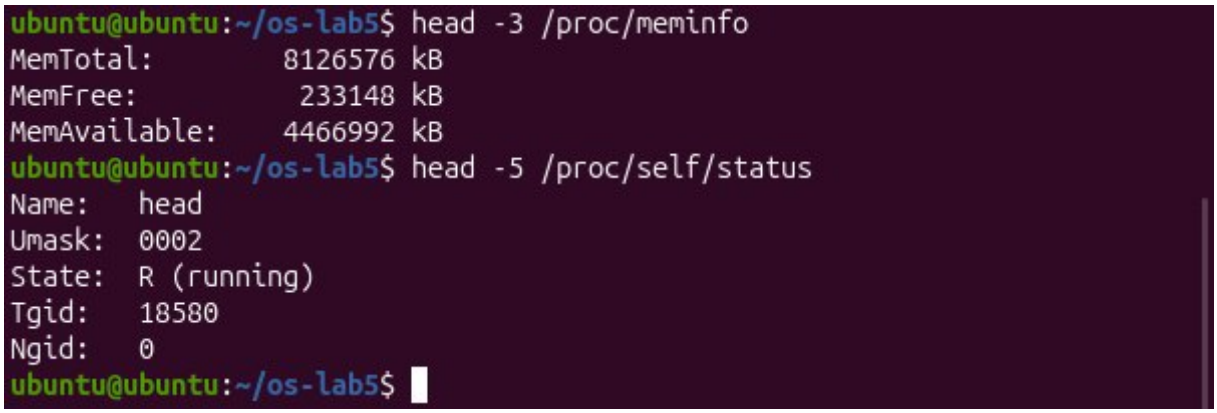


Figure 4.4: The memory information and the status of the process `head`

head -3 /proc/meminfo – prints the first three lines of the memory information of the kernel: `MemTotal` 8126576 kB, `MemFree` 233148 kB and `MemAvailable` 4466992 kB (Figure 4.4). The file is not stored on a disk; the kernel generates its text at the moment of reading.

head -5 /proc/self/status – prints the first five lines of the status of the process that reads the file: `Name: head`, `Umask: 0002`, `State: R (running)`, `Tgid: 18580` and `Ngid: 0`.

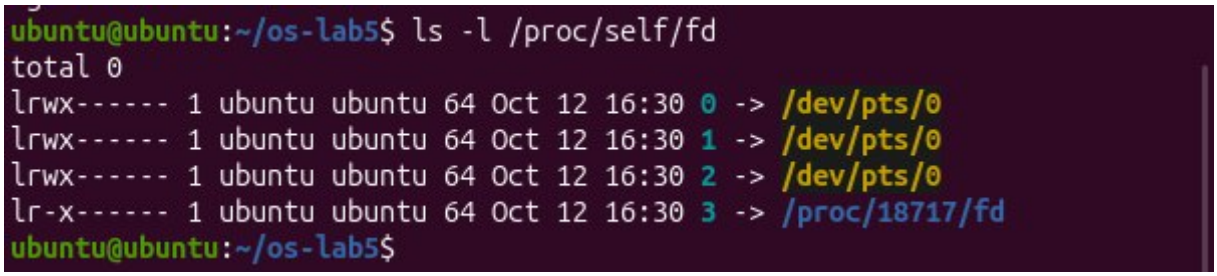


Figure 4.5: The open file descriptors of the process `ls`

ls -l /proc/self/fd – lists the files opened by the process `ls` itself. In Figure 4.5 the descriptors 0, 1 and 2 (the standard input, output and error) are links to `/dev/pts/0`, the terminal. The descriptor 3 is a link to `/proc/18717/fd`, the directory that `ls` opened in order to read it, and 18717 is the number of the `ls` process.

Device	**Size**	**Remarks**	**Mounted on**
`/dev/sr0`	6.1G	installation image, 100%	`/cdrom`
`loop0`	3.2G	read-only	`/rofs`
`loop1`	562.7M	read-only	–
`loop2`	756.4M	read-only	–
`loop3`	66.8M	read-only, snap	`/snap/core24/1643`
`loop4`	20M	read-only, snap	`/snap/desktop-security-center/151`
`loop5`	4K	read-only, snap	`/snap/bare/5`
`/cow` (overlay)	3.9G	478M used, 13%	`/`

Table 4: The devices and the mount points of the system

Table 4 summarizes the devices. The root directory is an overlay of read-only images and a writable layer, and the files of `/proc` show that the kernel presents its own state, such as the processors, the memory and the open descriptors of a process, through the same interface of files.

Conclusion

The laboratory work explored the Linux file system with ordinary commands and careful observation. In the first part, the type of each path was read from the first letter of `ls -l`: directories, a regular file, a character device and the symbolic link `/bin -> usr/bin`, and the sizes of `/var/log` (21M) and `/usr` (5.3G) were compared. In the second part, `stat` and `ls -li` showed that the name of a file is not part of its inode: a hard link gave the inode 19896 a second name, and the file survived `rm notes.txt`, because `rm` removes only a name and decreases the link count, while a symbolic link, with its own inode and a size of 8 bytes, broke as soon as its target disappeared. The link counts of directories (2 and 4) and the sparse file of 1 GB, which used no blocks, showed the difference between the size of a file and the space it occupies.

The third part showed that access is decided by the permission bits of the class that matches the user: `chmod 000` blocked even the owner, the `umask` 0002 produced the modes 664 for files and 775 for directories, `r` and `x` on a directory allow listing the names and passing through it, and the bits `s` and `t` explain how `passwd` can change `/etc/shadow` and why users cannot delete the files of each other in `/tmp`. In the last part, `lsblk`, `df`, `mount` and the files of `/proc` described the system, a live session with an overlay root file system (13% used), 4 processors and about 8 GB of RAM, and proved that the kernel exposes its state as files, as in `/proc/self`, which always refers to the process that reads it. Overall, the laboratory work gave a practical understanding of inodes, links, permissions and the file interfaces of the Linux kernel, and all the objectives of the work were achieved.

References

1. *Gabriela Pricop. os-2026-GabrielaPricop: laboratory works of the Operating Systems course, GitHub repository*
   `https://github.com/gabriela-pricop/os-2026-GabrielaPricop`
2. *inode(7) – Linux manual page: inode number, link count, file type and mode bits*
   `https://manpages.debian.org/unstable/manpages/inode.7.en.html`
3. *chmod(1) – change file mode bits, GNU coreutils manual page*
   `https://manpages.debian.org/buster/coreutils/chmod.1`
4. *proc(5) – the process information pseudo-file system, Linux manual page*
   `https://linux.die.net/man/5/proc`